University of Padova

Department of Mathematics "Tullio Levi-Civita"

Master Thesis in Computer Science

Crossdem: TODO

Master Candidate
Marco Bellò

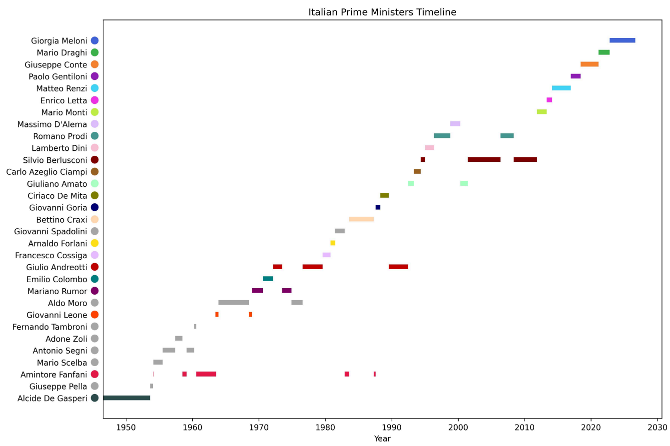
Supervisor
Prof. Alessandro Galeazzi
University of Padova

Student ID
2122196

Academic Year
2026-2027

Marco Bellò, *Crossdem: TODO*
Master Thesis in Computer Science , University of Padova, © 2026-2027

This document was prepared using Typst, based on Davide Spada's template. In addition, LLMs (Claude Sonnet 5, Gemini 3.6 Flash, and Copilot Free) were used to check both syntax and semantics. All source code (Python and Typst), as well as the whole dataset, are available in the repository:
https://github.com/mhetacc/crossdem



Look again at that dot. That's here. That's home. That's us.
On it everyone you love, everyone you know, everyone you ever heard of, every human being who ever was, lived out their lives.
The aggregate of our joy and suffering, thousands of confident religions, ideologies and economic doctrines, every hunter and forager, every hero and coward, every creator and destroyer of civilization, every king and peasant, every young couple in love, every mother and father, hopeful child, inventor and explorer, every teacher of morals, every corrupt politician, every "superstar", every "supreme leader", every saint and sinner in the history of our species lived there–on a mote of dust suspended in a sunbeam.
— Carl Sagan, *Pale Blue Dot*, 1994

Abstract

In the past decades, since the introduction of the Internet of Things (IoT) paradigm, the number of connected devices has grown exponentially. These applications now dominate our everyday life across multiple domains, such as smart homes, healthcare or industrial automation.
This growth has proceeded hand in hand with the development of various protocols to support different requirements. However, this technological variety introduces significant architectural fragmentation, which may limit interoperability. The nature of some IoT applications has raised significant security and privacy concerns, especially with regard to sensitive data or critical infrastructures, where a lack of security measures and cyber-attacks could have severe consequences. Therefore, a risk analysis is essential to identify vulnerabilities and design defence effective strategies.
In this thesis, we present a multi-protocol IoT framework designed to address the challenge of protocol fragmentation by providing a unified, protected communication environment. The work focuses on the design, implementation and security analysis of the two most widely adopted IoT protocols: MQTT and CoAP. This framework is applied to the specific use case of smart agriculture, with a focus on monitored and automated greenhouses. The core of the study involves addressing potential cyber threats by implementing appropriate security mechanisms, such as authentication, authorization and encryption, to mitigate risks without compromising the efficiency required by agricultural sensors.
The work demonstrates the challenges of integrating heterogeneous IoT protocols within a secure framework, highlighting the importance of addressing both interoperability and security.
This project was carried out during the internship at M31 S.r.l., an IoT solutions company based in Padova.

Contents

Abstract iv

List of Tables vii

List of Figures viii

List of Codes xi

List of Acronyms xii

1 Introduction 1
- 1.1 Research Questions ... 1
- 1.2 Roadmap ... 2

2 State of the Art 3
- 2.1 Textual Complexity ... 3
- 2.2 Polarization ... 3
- 2.3 Sentiment Analysis ... 3

3 The Corpus 5
- 3.1 Similar Corpora ... 5
- 3.2 My Corpus ... 5
- 3.3 Building the Corpus ... 7
  - 3.3.1 De Gasperi ... 7
  - 3.3.2 Scraping ... 7
  - 3.3.3 Corpus Annotation ... 7
  - 3.3.4 V-DEM ... 8

4 Results 9
- 4.1 Textual Complexity ... 9
  - 4.1.1 Measure of Textual Lexical Diversity ... 9
  - 4.1.2 Prime Minister's MTLDs ... 10
  - 4.1.3 VDEM ... 12
  - 4.1.4 Conclusions ... 14
- 4.2 Polarization ... 15

4.2.1 Prime Minister's Lexical Similarity ... 15
4.2.2 Political Leanings Lexicons Over Time ... 16
4.2.3 t-SNE ... 17
4.2.4 Conclusions ... 18
4.3 Sentiment Analysis ... 19
4.3.1 Democratic Indices Against Sentiment Scoring ... 19
4.3.2 Toxicity (TODO: change name) ... 19
4.3.3 Targets ... 20
4.3.4 Conclusions ... 21

5 Discussion & Future Works 25

A Appendix 26
1.1 Tables ... 26
1.1.1 Not Colored ... 26
1.1.2 Colored ... 27
1.1.3 Half Colored Half Not ... 27
1.1.4 Stacked ... 28
1.2 Figures ... 28
1.2.1 Single Figure ... 28
1.2.2 Double Figures ... 29
1.3 Code Listings ... 29

References 33

List of Tables

Table A.1 10-level topic hierarchy structure 26
Table A.2 10-level topic hierarchy structure 27
Table A.3 10-level topic hierarchy structure 27
Table A.4 Sample rows from `iot_records_tenant_01` (top) and `iot_records_tenant_02` (bottom). . 28

List of Figures

Figure 3.1 Italian Prime Ministers from 1946 to 2025. In grey Prime Ministers not included in the corpus. 6

Figure 2 Figure 2a show the number of words collected for the dataset over the span of eighty years. In green are words pertaining to center-leaning Prime Ministers, in red are words pertaining to left-leaning Prime Ministers, and in blue are words pertaining to right-leaning Prime Ministers. Figure 2b, on the other hand, shows the same data but in terms of tokens instead of whole words. 7

Figure 3 Figure 3a and Figure 3b show MTLD values for each Italian Prime Minster, ordered by their first time in office. This allows for not only a direct comparison between Prime Ministers, but also a comparison of the evolution of MTLD values over time. Prime Minister with fewer than 100 speeches are omitted. 10

Figure 4 Figure 4a and Figure 4b show MTLD values for each Italian Prime Minster over their careers span. The LOWESS curve allows was chosen to aid in the interpretation of the values, and the overall medians allow for a comparison between Prime Ministers. Prime Minister with fewer than 100 speeches are omitted. 11

Figure 5 MTLD values in a Boxplot by 5-year periods. There seem to be a reduction in MTLD variance starting from 1989 all the way to 2019, whith an abrupt inversion in the 2020-2025 period. If this is a correlation with the inflection of some VDEM indices, it is not clear. 12

Figure 6 MTLD values in a Boxplot by 10-year periods. There seem to be a reduction in MTLD variance starting from 1989 all the way to 2025. If this is a correlation with the inflection of some VDEM indices, it is not clear. 13

Figure 7 MTLD values aggregated for each year, plotted against VDEM indices. There does not seem to be a correlation between a decrease in complexity and a decrease in democratic indices. Instead, the graph seems to suggest the opposite. 14

Figure 8 Heatmaps showing lexical similarity between Prime Ministers' speeches. Figure 8a was made by pooling the 1000 most used words by each Prime Minister, then calculating how many words are shared between each of them. Figure 8b, on the other hand, is made by calculating the Jaccard index between each pair of Prime Ministers, by pooling *all* words from all speeches used by each Prime Minister. 15

Figure 9 All speeches are pooled in 5-year time frames. Then, for each 5-year time frame, the top-N most frequently used words are extracted for each political leaning[1] (one thousands words for graph at Figure 9a, and ten thousands words for graph at Figure 9b). This results in three distinct sets[2] of words for left, right, and center leaning speeches respectively.

[1] A speech is considered "right-leaning" if it pertains a right-leaning Prime Minister.

[2] When speeches for each leaning are present. Otherwise only two, one, or no set at all can be extracted from that 5-year time frame.

The graph shows, for each 5-year time frame, words in common among all leanings (in grey), and words unique for each leaning (in red, blue, and green respectively). For the years with most data (from 1985 to 2015), Figure 9a shows a progressive increase in unique vocabularies compared to a decrease in shared ones, which can be interpreted as symptoms of increasing polarization. 16

Figure 4.10 Top-1000 words[3] from each Prime Minster. The embedding vectors are pretrained Italian word vectors (CBOW, 300 dimensions) taken from *fastText*. Words shared by all Prime Ministers are displayed as black rhombuses. The graph show a lot of clustering, suggesting a certain similarity among Prime Ministers. That being said, the graph is also too hard to read to gain any conclusive evidence. 17

Figure 11 Using `TfidfVectorizer` from `sklearn` library, I computed one embedding for each speech[4]. The embeddings can then be projected in two dimensions. Figure 11a shows the projected speech vectors grouped by Prime Minsters. We can see clustering, suggesting that most prime ministers have a lexical style and wording that are unique to them. Figure 11b groups the vectors by political leaning, showing that different leanings occupy different and distinct regions in the vector space. Both graphs strongly hints to a polarized political landscape.

On the other hand, Figure 11c shows two main clustering regions: on the top, speeches from 1945 to 1955 gather, while speeches from 1991 to 2025 are more uniformly scattered across the whole vector space. This suggests that earlier speeches were significantly different compared to modern ones, while speeches from the nineties onward are quite similar. 18

Figure 12 Sentiment analysis results for *hate speech*, *aggressiveness*, and *negativity* plotted against VDEM indices. Its hart to tell wether there is any correlation between them. 19

Figure 13 Sentiment analysis results for *negativity* and *aggressiveness*. Values go from zero to one, and the classification task considers three possibile values for each speech: *low* (0), *mid* (0.5), and *high* (1).

Size of bubbles depends on the amount of speeches. Figure 13a project results for single Prime Ministers: Berlusconi stands out as the most aggressive by far. Figure 13b project results aggregated by leanings. The right appears to be more aggressive, while the left more negative. The center seems to be more positive than both. The last two figures show the sentiment scores aggregated by years (with 5-year pool range). While Figure 13c shows an outlier for the period $1976 - 1980$, Figure 13d shows that overall the sentiment scores are quite close to each other, without great variance especially for periods with a lot of speeches. 20

Figure 14 All graphs show the percentage of speeches that target political adversaries. Figure 14a shows which Prime Minister speaks more often about the opposition (prime ministers with less than 100 speeches are omitted). Figure 14b shows in which 5-year periods there are more speeches targeting political adversaries (ordered by year). Lastly, Figure 14c

[3] Top-1000 words means pooling the one thousand most used words of a specific Prime Minister. Stopwords are removed and the lexicon is lemmatized.

[4] Each speech counts as a document.

shows which of the three political fields is more likely to speak about the opposing front. The results show how, across the board, more than one speech out of two targets the opposition. This is true regardless of time period or affiliation. 21
Figure 15 Overall, an extremely low percentage of speeches target minority groups. Therefore, I will ignore this analysis. (TODO explain better.. within error yada yada). 21
Figure A.16 Architecture of IoT (A: 3-layers) (B: 5-layers) 28
Figure A.17 Comparison between (a) Request-Response and (b) Publish-Subscribe paradigms 29

List of Codes

Code A.1 Example of a Permify tuple 29
Code A.2 Example of a Permify tuple 29
Code A.3 Permify schema example 30
Code A.4 Example of a permission check request to Permify 30
Code A.5 VerneMQ configuration in Docker Compose - relevant parts 31
Code A.6 Auth service - webhook endpoints handlers 31
Code A.7 NATS JetStream stream configuration 32
Code A.8 DTLS configuration with PSK callback function 32
Code A.9 DB Writer input payload structure 32

List of Acronyms

MQTT................................Message Queuing Telemetry Transport

CoAP..................................Constrained Application Protocol

IoT......................................Internet of Things

BLE.....................................Bluetooth Low Energy

LoRa..................................Long Range

LoRaWAN.........................Long Range Wide Area Network

NB-IoT...............................Narrowband Internet of Things

AES-CCM..........................Advanced Encryption Standard - Counter with CBC-MAC

MAC..................................Message Authentication Code

TCP....................................Transmission Control Protocol

mTLS.................................Mutual Transport Layer Security

M2M..................................Machine to Machine

IAM....................................Identity Access Management

HA......................................High Availability

IDS.....................................Intrusion Detection Systems

ReBAC...............................Relationship-Based Access Control

1
Introduction

- Importance of political speeches
- Public vs institutional speeches
- External speeches
    - easier scraping
        - social networks
        - internet
    - LLMs trained on common speech
- Textual complexity, polarization, and sentiment analysis
    - Why are they important

1.1 Research Questions

Research Questions:

1. **Textual Richness:**
    1.1. Does it have a correlation with democracy levels?
    1.2. Does it change over time?
    1.3. Does it change among different political leanings?
2. **Polarization:**
    2.1. Does symptoms of polarization change over the years?
    2.2. Do they have a correlation with democracy levels?
    2.3. Are Prime Ministers' rhetorics more similar or more dissimilar?
    2.4. Can we visualize and calculate polarization among different political leanings?
3. **Sentiment:**
    3.1. Does it have a correlation with democracy levels?

3.2. What PM is more toxic?
3.3. What political leaning is more toxic?
3.4. What period is more toxic? Is it getting better or worse?
3.5. Target:
    3.5.1. Which political side, if any, is more often targeting?
    3.5.2. Which PMs, if any, si more often targeting?

1.2 Roadmap

Needed?

1. First step: build the corpus
2. Second step: etc

2
State of the Art

2.1 Textual Complexity

- What is
- Common metrics
- State of the art: what has been done in the field, what are the gaps, what are the limitations

2.2 Polarization

- What is
- Common metrics
- State of the art: what has been done in the field, what are the gaps, what are the limitations

2.3 Sentiment Analysis

- What is
- Common metrics
- State of the art: what has been done in the field, what are the gaps, what are the limitations

3
The Corpus

3.1 Similar Corpora

3.2 My Corpus

- Structure
- Content
    - prime ministers
    - transcriptions
    - annotations

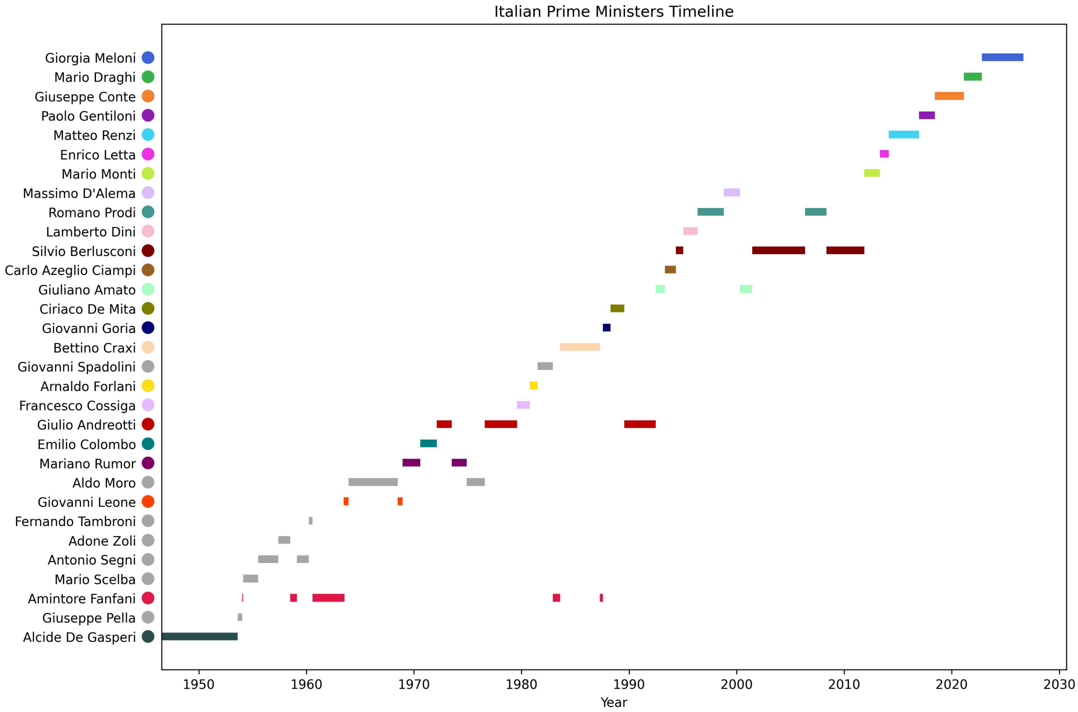
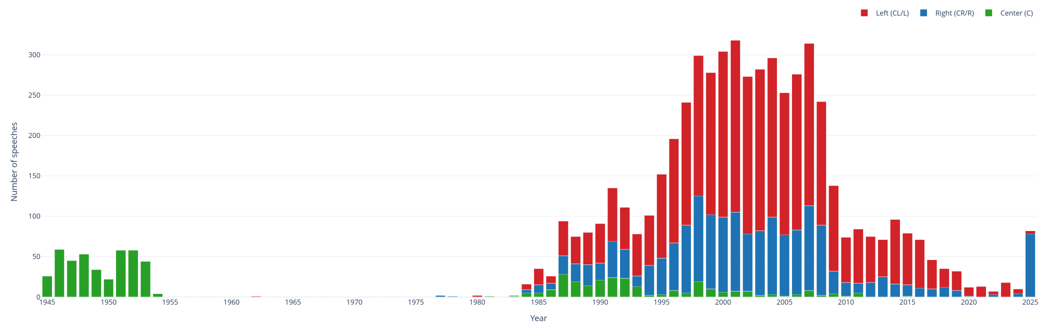
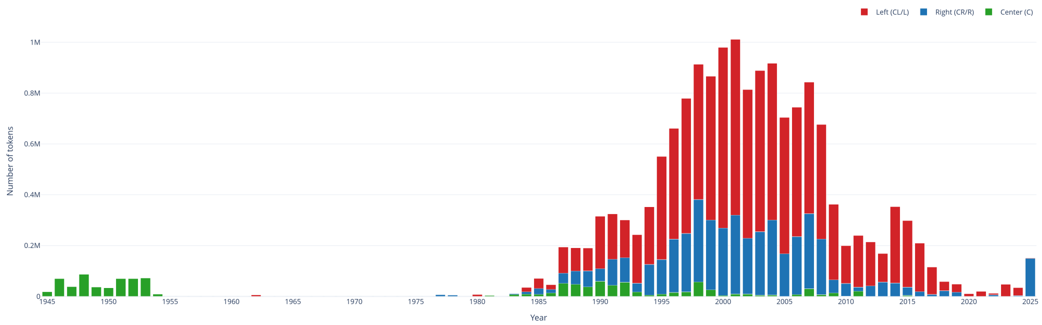


Figure 3.1: Italian Prime Ministers from 1946 to 2025. In grey Prime Ministers not included in the corpus.



(a) Dateset: words. The graph show strong data imbalances: years between 1995 and 2010 contain most of the data, and left-leaning Prime Ministers are overrepresented.



(b) Dataset: tokens. Tokenization done via `spacy.load("it_core_news_sm")`.

Figure 2: Figure 2a show the number of words collected for the dataset over the span of eighty years. In green are words pertaining to center-leaning Prime Ministers, in red are words pertaining to left-leaning Prime Ministers, and in blue are words pertaining to right-leaning Prime Ministers. Figure 2b, on the other hand, shows the same data but in terms of tokens instead of whole words.

3.3 Building the Corpus

3.3.1 De Gasperi

Thanks to Tonelli et. al. [1] for the De Gasperi corpus.

3.3.2 Scraping

- Scraping
  - Meloni YouTube
  - Radio Radicale
- Speech-to-text via OpenAI *Whisper*

3.3.2.1 Scraping Sanity Check

Is the corpus correct? Yes, check with n-gram centroid.

3.3.3 Corpus Annotation

- Choosing how to annotate the corpus
  - Choosing the right LLMs

- Pipeline

3.3.4 V-DEM

4 Results

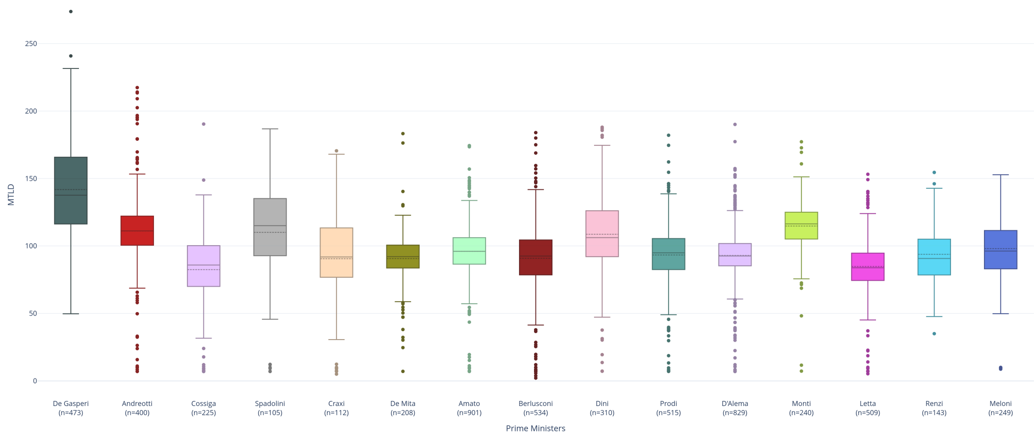
4.1 Textual Complexity

4.1.1 Measure of Textual Lexical Diversity

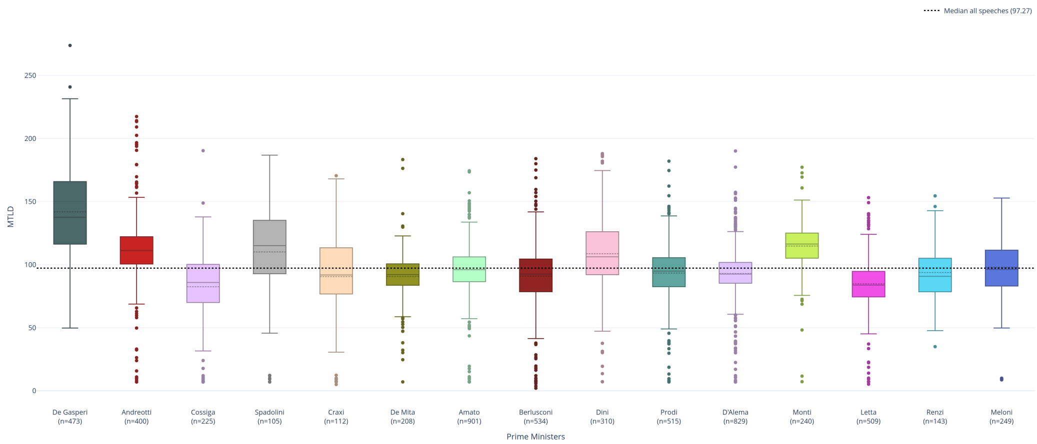
Measure of Textual Lexical Diversity (MTLD) is a metric used to assess the diversity of vocabulary in a text. It calculates the average length of sequences of words that maintain a certain level of lexical diversity, providing insights into the richness and variety of language used in the text. yada yada

4.1.2 Prime Minister's MTLDs

4.1.2.1 Boxplot of MTLD Values for Each Italian Prime Minster



(a) Boxplot of MTLD values for each Italian Prime Minster, without overall median.

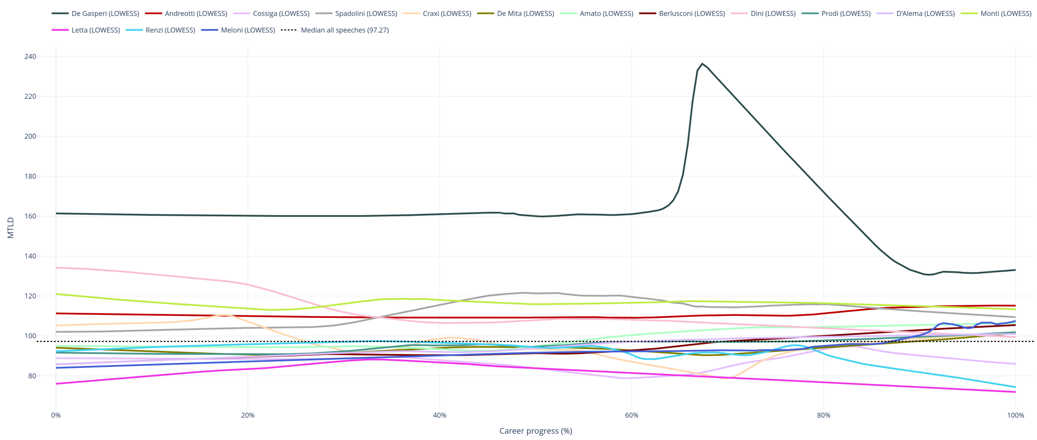


(b) Boxplot of MTLD values for each Italian Prime Minster, with overall median.

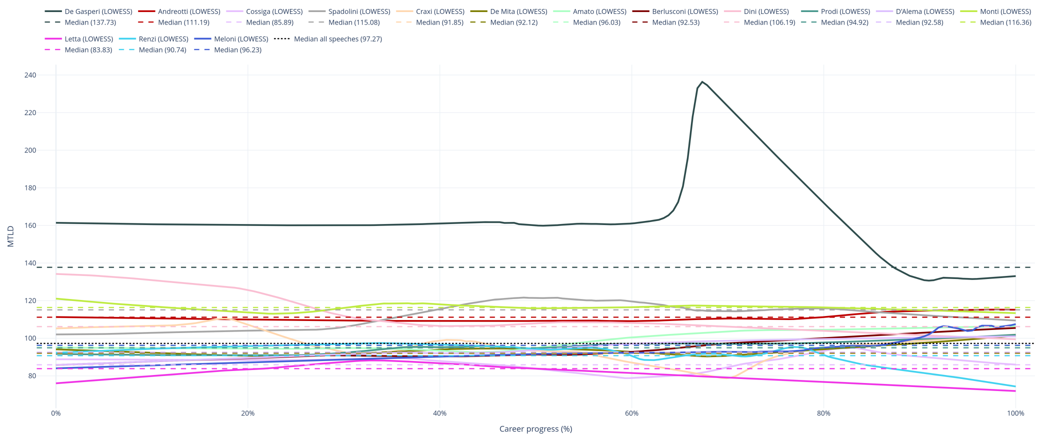
Figure 3: Figure 3a and Figure 3b show MTLD values for each Italian Prime Minster, ordered by their first time in office. This allows for not only a direct comparison between Prime Ministers, but also a comparison of the evolution of MTLD values over time. Prime Minister with fewer than 100 speeches are omitted.

4.1.2.2 Evolution Over Time of MTLD Values for Each Italian Prime Minster

- what is LOWESS



(a) LOWESS curve of MTLD values for each Italian Prime Minster, without overall medians.

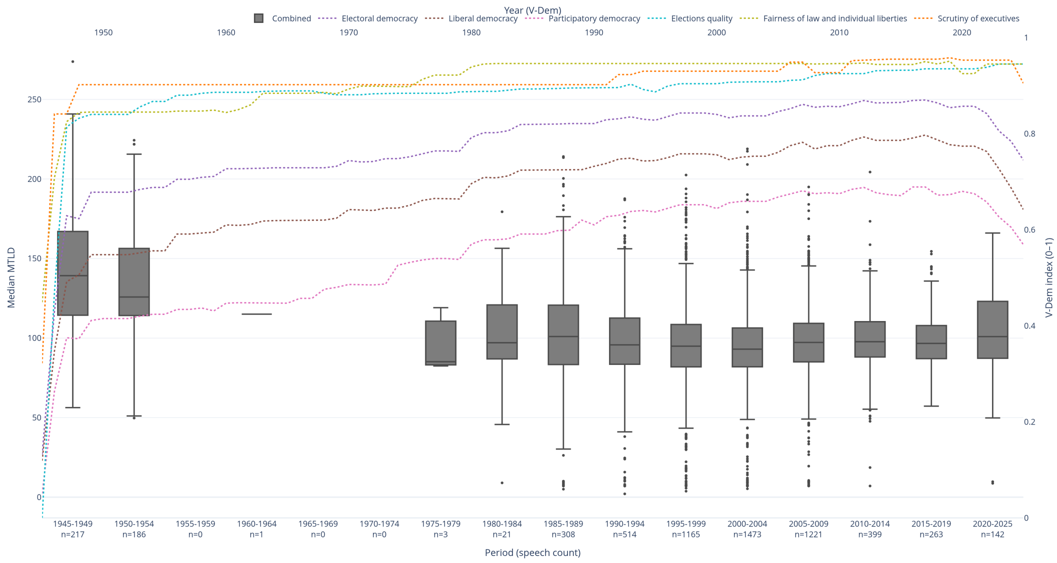


(b) LOWESS curve of MTLD values for each Italian Prime Minster, with overall medians.

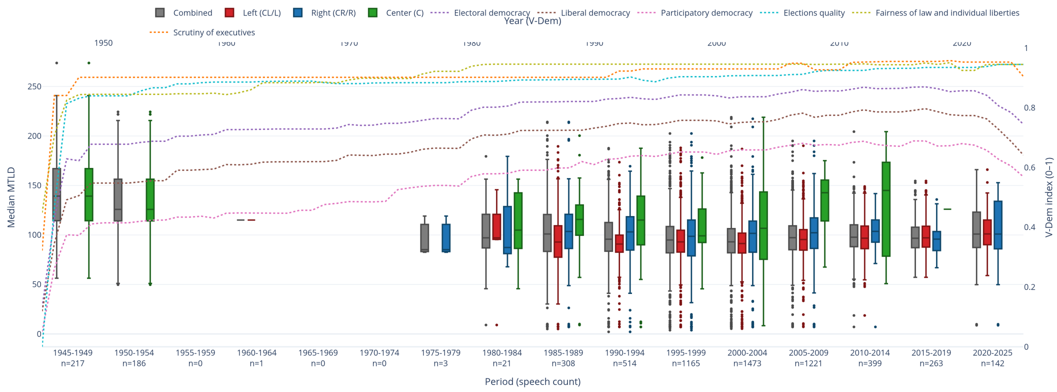
Figure 4: Figure 4a and Figure 4b show MTLD values for each Italian Prime Minster over their careers span. The LOWESS curve allows was chosen to aid in the interpretation of the values, and the overall medians allow for a comparison between Prime Ministers. Prime Minister with fewer than 100 speeches are omitted.

4.1.3 VDEM

4.1.3.1 5 YEARS



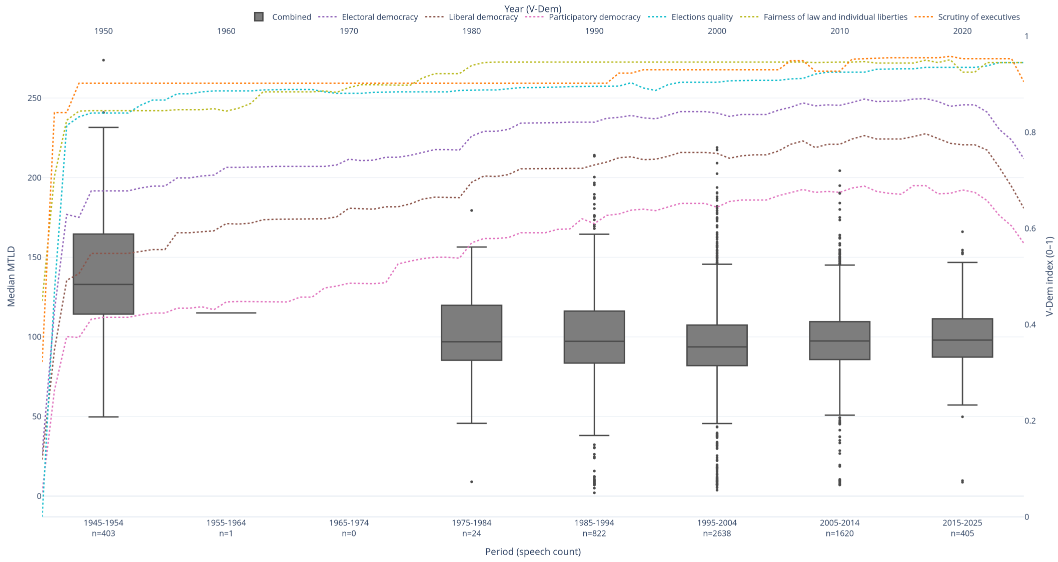
(a) MTLD values in a Boxplot by 5-year periods, with VDEM indices. Each grey box combines all speeches for all political leanings in that 5-year period.



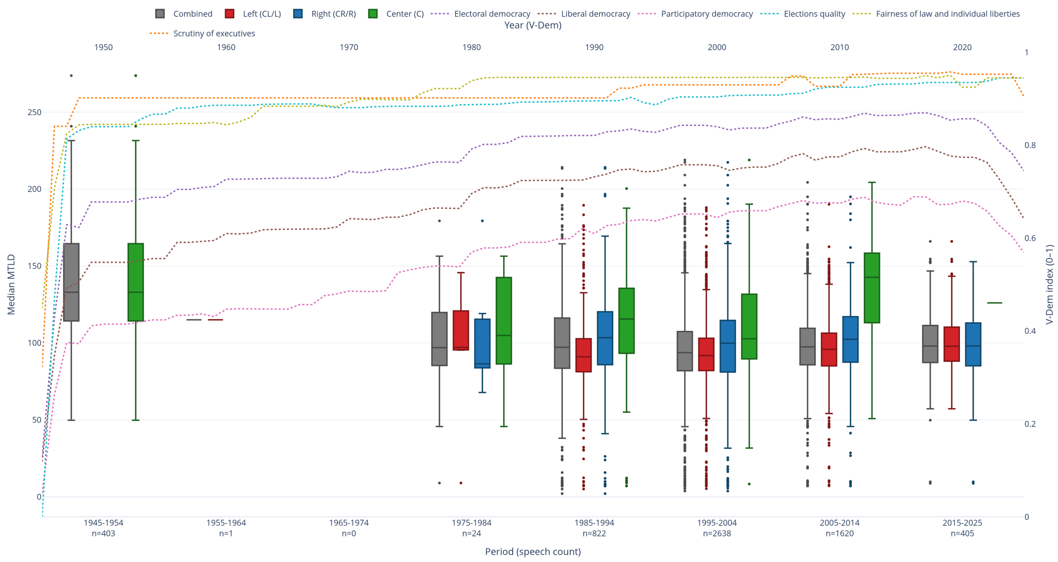
(b) MTLD values in a Boxplot by 5-year periods, with VDEM indices. As above, each grey box combines all speeches for all political leanings in that 5-year period, while the colored boxes represent respectively center-leaning (in green), left-leaning (in red), and right-leaning (in blue) MTLD values in that 5-year preiod.

Figure 5: MTLD values in a Boxplot by 5-year periods. There seem to be a reduction in MTLD variance starting from 1989 all the way to 2019, whith an abrupt inversion in the 2020-2025 period. If this is a correlation with the inflection of some VDEM indices, it is not clear.

4.1.3.2 10 YEARS



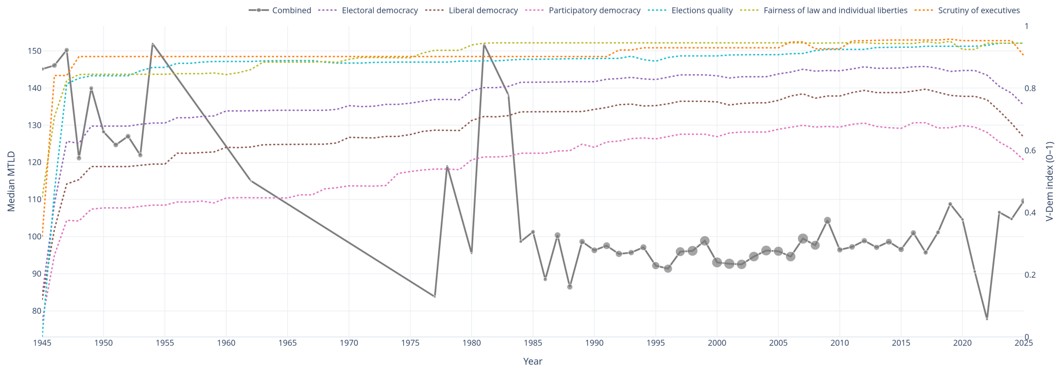
(a) MTLD values in a Boxplot by 10-year periods, with VDEM indices. Each grey box combines all speeches for all political leanings in that 10-year period.



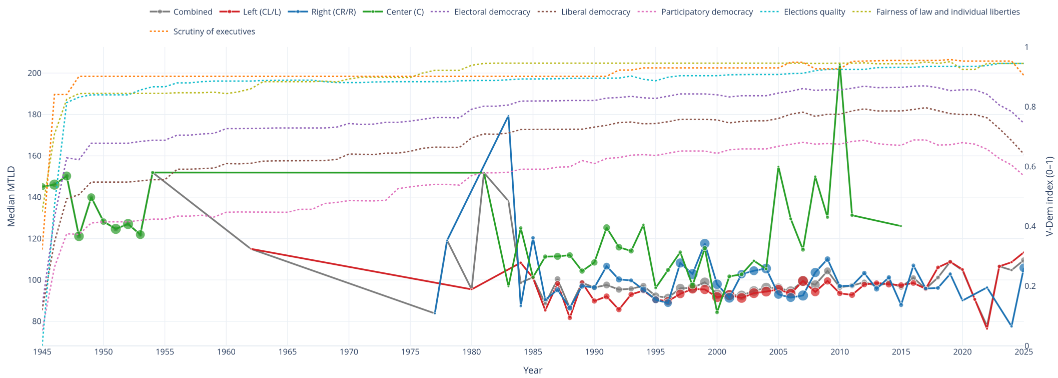
(b) MTLD values in a Boxplot by 10-year periods, with VDEM indices. As above, each grey box combines all speeches for all political leanings in that 10-year period, while the colored boxes represent respectively center-leaning (in green), left-leaning (in red), and right-leaning (in blue) MTLD values in that 10-year preiod.

Figure 6: MTLD values in a Boxplot by 10-year periods. There seem to be a reduction in MTLD variance starting from 1989 all the way to 2025. If this is a correlation with the inflection of some VDEM indices, it is not clear.

4.1.3.3 All



(a) MTLD values plotted for each year, aggregating all speeches from all political leanings. Each dot of the graph (for each year) is scaled based on the amount of speeches available for that specific year. The MTLD line chart is plotted against VDEM indices.



(b) MTLD values plotted for each year, aggregating speeches from three political leanings, respectively colored as green (center), red (left), and blue (right). Each dot of the graph (for each year) is scaled based on the amount of speeches available for that specific year. The MTLD line chart is plotted against VDEM indices.

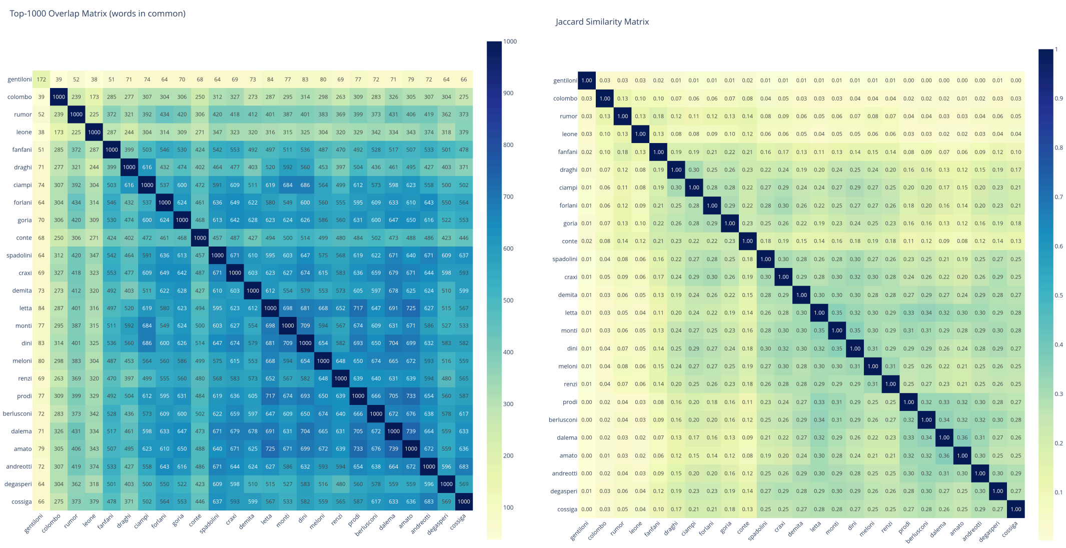
Figure 7: MTLD values aggregated for each year, plotted against VDEM indices. There does not seem to be a correlation between a decrease in complexity and a decrease in democratic indices. Instead, the graph seems to suggest the opposite.

4.1.4 Conclusions

There does not seem to be a clear correlation between MTLD values and democratic indices, with the exception for the variance which ...

4.2 Polarization

4.2.1 Prime Minister's Lexical Similarity

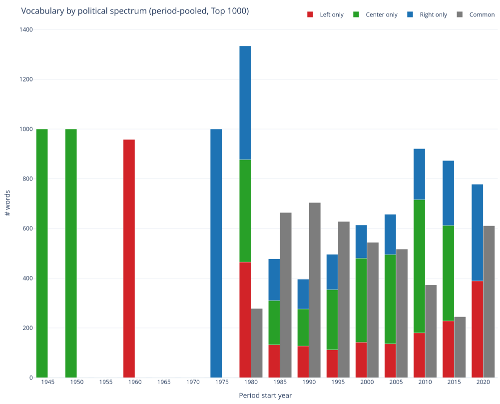


(a) Words similarity.

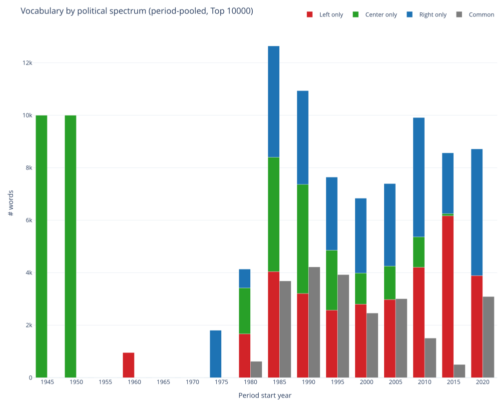
(b) Jaccard similarity.

Figure 8: Heatmaps showing lexical similarity between Prime Ministers' speeches. Figure 8a was made by pooling the 1000 most used words by each Prime Minister, then calculating how many words are shared between each of them. Figure 8b, on the other hand, is made by calculating the Jaccard index between each pair of Prime Ministers, by pooling *all* words from all speeches used by each Prime Minister.

4.2.2 Political Leanings Lexicons Over Time



(a) Top 1, 000 pooling.



(b) Top 10, 000 pooling

Figure 9: All speeches are pooled in 5-year time frames. Then, for each 5-year time frame, the top-N most frequently used words are extracted for each political leaning[5] (one thousands words for graph at Figure 9a, and ten thousands words for graph at Figure 9b). This results in three distinct sets[6] of words for left, right, and center leaning speeches respectively.

The graph shows, for each 5-year time frame, words in common among all leanings (in grey), and words unique for each leaning (in red, blue, and green respectively). For the years with most data (from 1985 to 2015), Figure 9a shows a progressive increase in unique vocabularies compared to a decrease in shared ones, which can be interpreted as symptoms of increasing polarization.

[5] A speech is considered "right-leaning" if it pertains a right-leaning Prime Minister.

[6] When speeches for each leaning are present. Otherwise only two, one, or no set at all can be extracted from that 5-year time frame.

4.2.3 t-SNE

4.2.3.1 Word2Vec: Word Embeddings

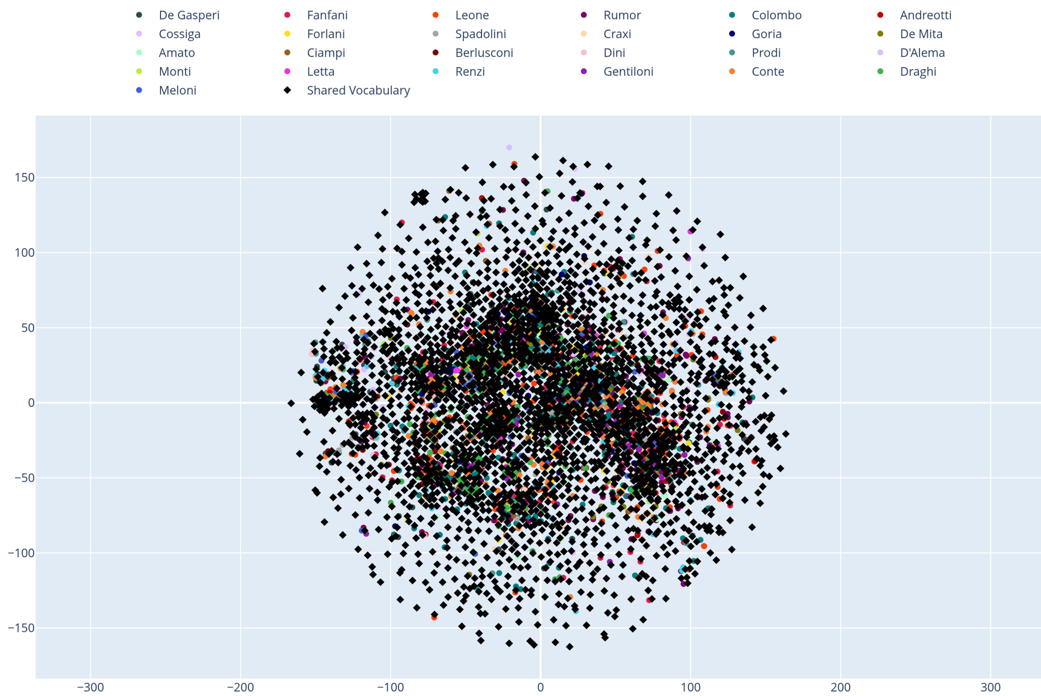
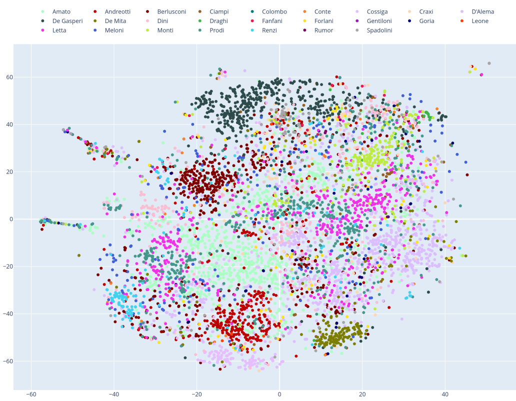


Figure 4.10: Top-1000 words[7] from each Prime Minster. The embedding vectors are pretrained Italian word vectors (CBOW, 300 dimensions) taken from *fastText*. Words shared by all Prime Ministers are displayed as black rhombuses. The graph show a lot of clustering, suggesting a certain similarity among Prime Ministers. That being said, the graph is also too hard to read to gain any conclusive evidence.

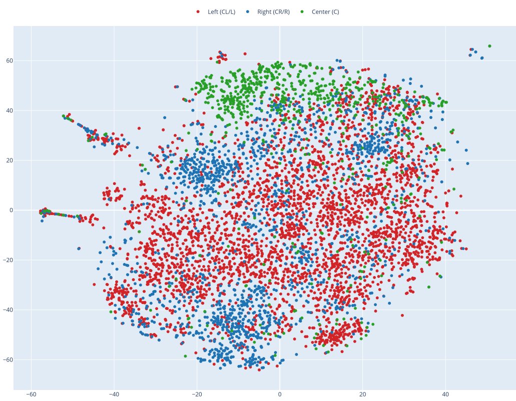
,

[7]Top-1000 words means pooling the one thousand most used words of a specific Prime Minister. Stopwords are removed and the lexicon is lemmatized.

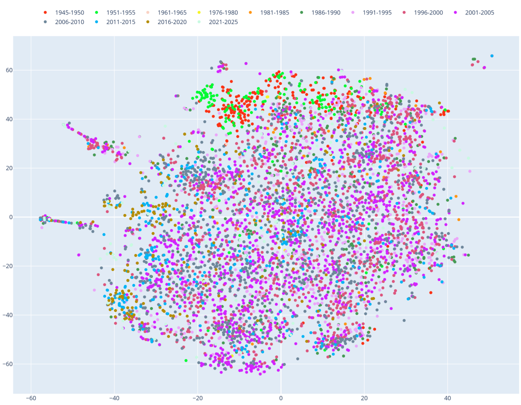
4.2.3.2 Doc2Vec: Speech Embeddings



(a) Doc2Vec: Prime Ministers



(b) Doc2Vec: political leaning



(c) Doc2Vec: year periods

Figure 11: Using `TfidfVectorizer` from `sklearn` library, I computed one embedding for each speech[8]. The embeddings can then be projected in two dimensions. Figure 11a shows the projected speech vectors grouped by Prime Minsters. We can see clustering, suggesting that most prime ministers have a lexical style and wording that are unique to them. Figure 11b groups the vectors by political leaning, showing that different leanings occupy different and distinct regions in the vector space. Both graphs strongly hints to a polarized political landscape.
On the other hand, Figure 11c shows two main clustering regions: on the top, speeches from 1945 to 1955 gather, while speeches from 1991 to 2025 are more uniformly scattered across the whole vector space. This suggests that earlier speeches were significantly different compared to modern ones, while speeches from the nineties onward are quite similar.

TODO: compare with VDEM polarization indices (if exists).

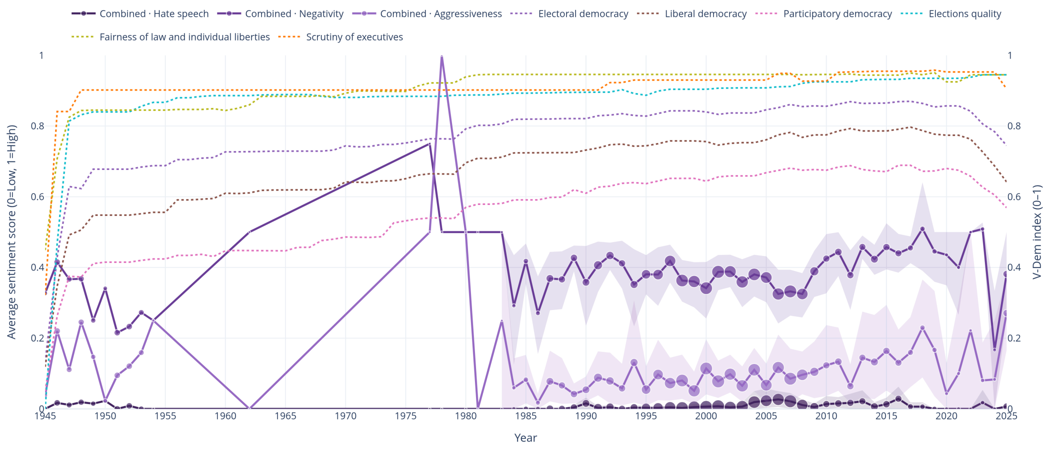
4.2.4 Conclusions

We see quite a bit of polarization, but ...

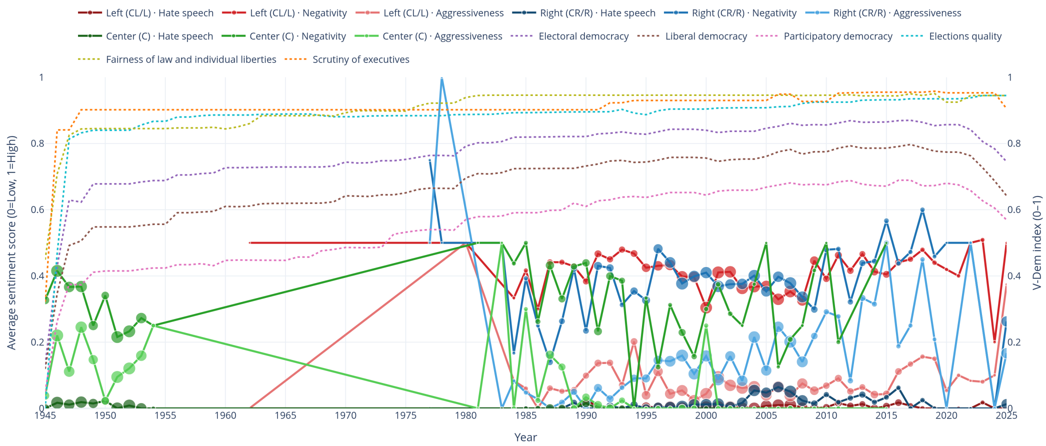
[8] Each speech counts as a document.

4.3 Sentiment Analysis

4.3.1 Democratic Indices Against Sentiment Scoring



(a) Sentiment values plotted for each year, aggregating all speeches from all political leanings. Each dot of the graph (for each year) is scaled based on the amount of speeches available for that specific year. The confidence intervals are shown. The sentiment scores chart is plotted against VDEM indices.

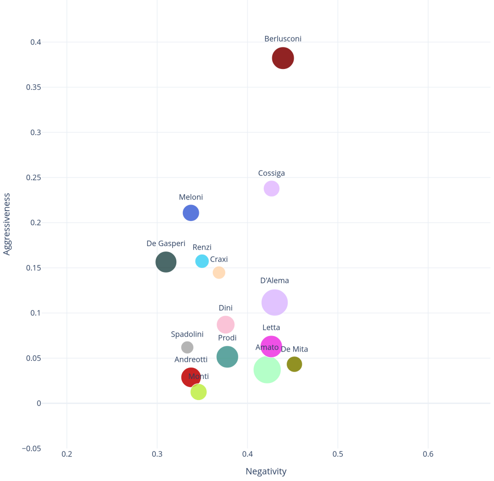


(b) Sentiment values plotted for each year, aggregating speeches from three political leanings, respectively colored as green (center), red (left), and blue (right). Each dot of the graph (for each year) is scaled based on the amount of speeches available for that specific year. The The sentiment scores line chart is plotted against VDEM indices.

Figure 12: Sentiment analysis results for *hate speech*, *aggressiveness*, and *negativity* plotted against VDEM indices. Its hart to tell wether there is any correlation between them.

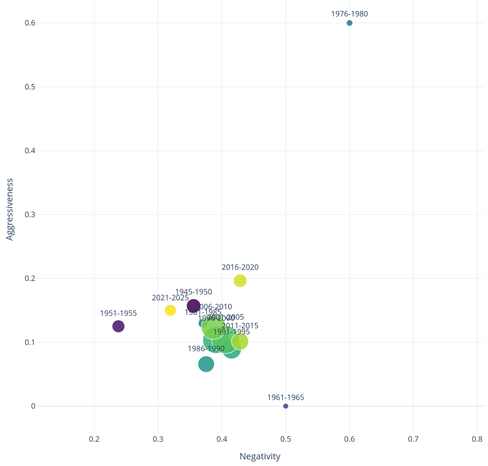
4.3.2 Toxicity (TODO: change name)

First thing first: hate speech is so low for everyone that we can ignore it



(a) Sentiment: Prime Ministers (|speeches| ≥ 100).

(b) Sentiment: leanings.



(c) Sentiment: years zoomed out.

(d) Sentiment: years zoomed in.

Figure 13: Sentiment analysis results for *negativity* and *aggressiveness*. Values go from zero to one, and the classification task considers three possibile values for each speech: *low* (0), *mid* (0.5), and *high* (1).
Size of bubbles depends on the amount of speeches. Figure 13a project results for single Prime Ministers: Berlusconi stands out as the most aggressive by far. Figure 13b project results aggregated by leanings. The right appears to be more aggressive, while the left more negative. The center seems to be more positive than both. The last two figures show the sentiment scores aggregated by years (with 5-year pool range). While Figure 13c shows an outlier for the period 1976 — 1980, Figure 13d shows that overall the sentiment scores are quite close to each other, without great variance especially for periods with a lot of speeches.

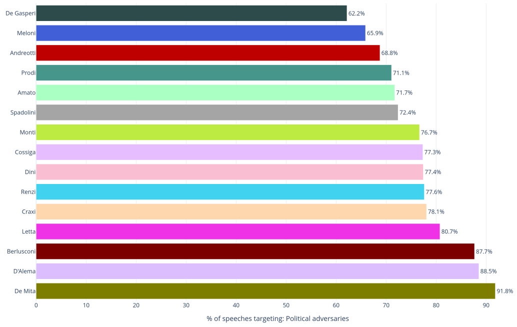
4.3.3 Targets

Five possible targets:

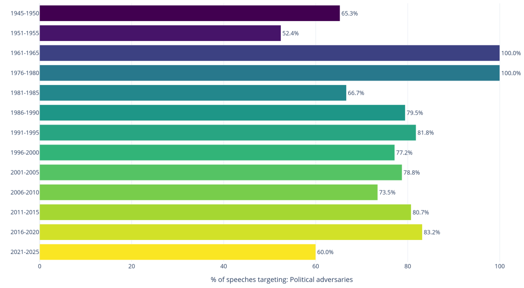
- None;
- Political adversaries;
- Gender minorities;

- Religious minorities;
- Ethnic minorities.

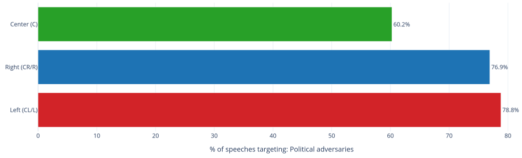
The targets are classified independently to the sentiment scores. For example, a speech could have *low* aggressiveness and still be targeting political adversaries.



(a) Target: political adversaries. Grouped by: PMs.



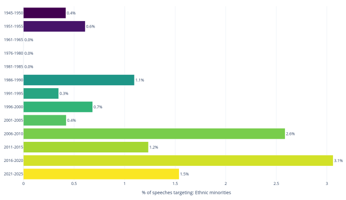
(b) Target: political adversaries. Grouped by: years.



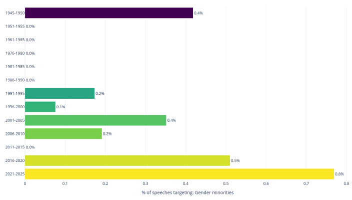
(c) Target: political adversaries. Grouped by: leanings.

Figure 14: All graphs show the percentage of speeches that target political adversaries. Figure 14a shows which Prime Minister speaks more often about the opposition (prime ministers with less than 100 speeches are omitted). Figure 14b shows in which 5-year periods there are more speeches targeting political adversaries (ordered by year). Lastly, Figure 14c shows which of the three political fields is more likely to speak about the opposing front. The results show how, across the board, more than one speech out of two targets the opposition. This is true regardless of time period or affiliation.

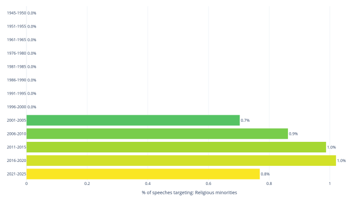
Thankfully, Italian Prime Minister apparently do not target minorities often.



(a) Target: ethnic minorities.



(b) Target: gender minorities.



(c) Target: religious minorities.

Figure 15: Overall, an extremely low percentage of speeches target minority groups. Therefore, I will ignore this analysis. (TODO explain better.. within error yada yada).

4.3.4 Conclusions

Italian prime minister seem to be overall a bit aggressive and negative, and they speak more often then not about the opposition. Almost no traces of hate speech and attacks against minority groups are found in the data.

This does not seem to change over time, failing to explain the VDEM indices drop in 2021.

5
Discussion & Future Works

- Analyses results
  - so can we explain VDEM drops and stuff?
- How to improve the work
  - more scraping (increase dataset)
  - better transcription (even tho they are pretty good)
  - bigger LLMs for the sentiment analysis
  - better doc2vec? gensim still not compatible with current Python version
- How to expand the work
  - include leader of opposition
  - include trend setter of the time
  - include other countries (eg France or USA)

A
Appendix

1.1 Tables

1.1.1 Not Colored

Level	Field	Description	Example
1	tenant_id	Logical isolation between organizations	agroTech01
2	site_type	Type of agricultural facility	greenhouse, open field
3	site_id	Physical site identifier	GH-001
4	section_id	Section within the site	section-2
5	zone_id	Operational zone	zone-A
6	entity_type	Device category	sensor, actuator
7	entity_subclass	Specific device type	temp, ph, water pump
8	entity_id	Unique device identifier	mqtt-device-03
9	message_type	Message classification	data, cmd, status, alert
10	target_id	Target device for commands (optional)	cooler-001

Table A.1: 10-level topic hierarchy structure

1.1.2 Colored

Level	Field	Description	Example
1	tenant_id	Logical isolation between organizations	agroTech01
2	site_type	Type of agricultural facility	greenhouse, open field
3	site_id	Physical site identifier	GH-001
4	section_id	Section within the site	section-2
5	zone_id	Operational zone	zone-A
6	entity_type	Device category	sensor, actuator
7	entity_subclass	Specific device type	temp, ph, water pump
8	entity_id	Unique device identifier	mqtt-device-03
9	message_type	Message classification	data, cmd, status, alert
10	target_id	Target device for commands (optional)	cooler-001

Table A.2: 10-level topic hierarchy structure

1.1.3 Half Colored Half Not

Level	Field	Description	Example
1	tenant_id	Logical isolation between organizations	agroTech01
2	site_type	Type of agricultural facility	greenhouse, open field
3	site_id	Physical site identifier	GH-001
4	section_id	Section within the site	section-2
5	zone_id	Operational zone	zone-A
6	entity_type	Device category	sensor, actuator
7	entity_subclass	Specific device type	temp, ph, water pump
8	entity_id	Unique device identifier	mqtt-device-03
9	message_type	Message classification	data, cmd, status, alert
10	target_id	Target device for commands (optional)	cooler-001

Table A.3: 10-level topic hierarchy structure

1.1.4 Stacked

Table iot_records_tenant_01

Time	Device ID	Tenant ID	Type	Data (JSONB)
2025-11-10 08:03:12 UTC	temp_001	tenant_01	data	{“value”: 22.4, “unit”: “C”}
2025-11-10 08:05:44 UTC	fan_003	tenant_01	status	{“running”: true, “speed”: “medium”}
2025-11-10 08:07:01 UTC	temp_001	tenant_01	alert	{“code”: “OVER_THRESHOLD”, “value”: 29.1}
2025-11-10 08:09:33 UTC	fan_003	tenant_01	cmd	{“action”: “set_speed”, “target”: “high”}

Table iot_records_tenant_02

Time	Device ID	Tenant ID	Type	Data (JSONB)
2025-11-10 08:10:22 UTC	temp_007	tenant_02	data	{“value”: 18.9, “unit”: “C”}
2025-11-10 08:10:25 UTC	air_008	tenant_02	data	{“value”: 412, “unit”: “ppm”}
2025-11-10 08:12:01 UTC	temp_007	tenant_02	status	{“online”: true, “battery”: 87}

Table A.4: Sample rows from `iot_records_tenant_01` (top) and `iot_records_tenant_02` (bottom).

1.2 Figures

`#align()` without any arguments defaults to *center*.

1.2.1 Single Figure

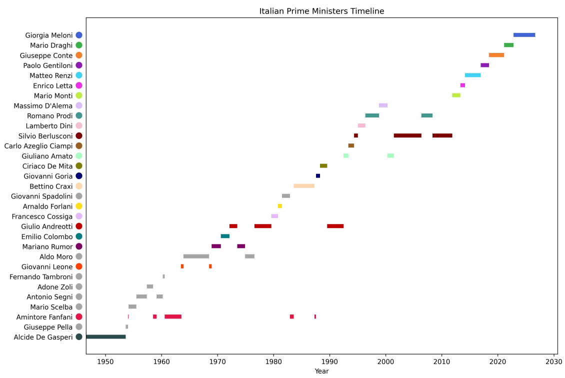


Figure A.16: Architecture of IoT (A: 3-layers) (B: 5-layers)

1.2.2 Double Figures

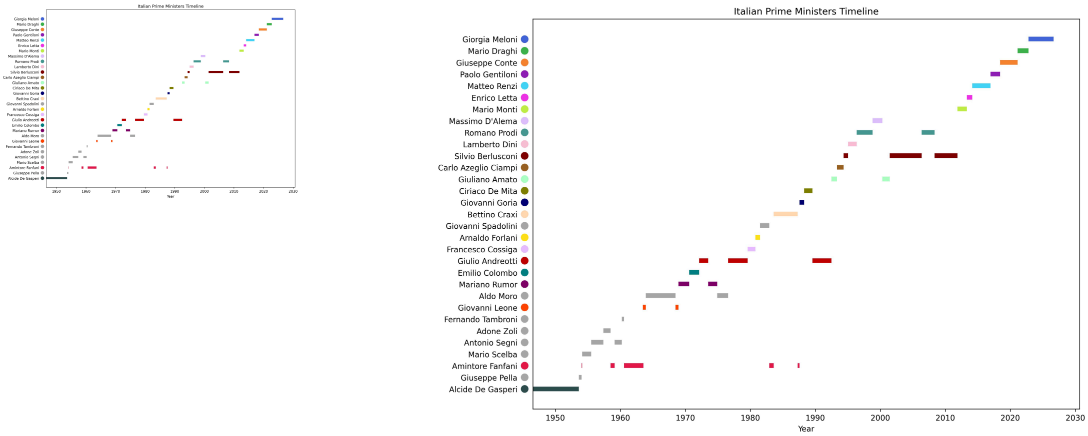


Figure A.17: Comparison between (a) Request-Response and (b) Publish-Subscribe paradigms

1.3 Code Listings

```
{
  "entity": { "type": "tenant", "id": "agroTech01" },
  "relation": "admin",
  "subject": { "type": "user", "id": "john_doe" }
}
```

Code A.1: Example of a Permify tuple

```python
def extract_page_date(soup: BeautifulSoup) -> tuple[date | None, str]:
    """Page-level event date (meta > title > stamp > sommario prose)."""
    for attr, val in [("name", "dcterms.date"), ("property", "article:published_time")]:
        tag = soup.find("meta", {attr: val})
        if tag and tag.get("content"):
            d = _try_iso(tag["content"])
            if d: return d, "meta_tag"

    title = soup.find("title")
    if title:
        d = _try_dotted(title.get_text())
        if d: return d, "page_title"

    page_text = soup.get_text(" ", strip=True)
    d = _try_short(page_text)
    if d: return d, "page_stamp"

    d = _try_long(page_text)
    if d: return d, "page_sommario"

    return None, "not_found"
```

Code A.2: Example of a Permify tuple

```
entity platform {
    relation super_admin @user
    permission manage_tenants = super_admin
}

entity tenant {
    relation admin      @user | @user_group#member
    relation maintainer @user | @user_group#member
    relation member     @user | @user_group#member

    permission is_admin      = admin
    permission is_maintainer = maintainer
    permission is_member     = member

    permission manage_structure = is_admin or is_maintainer
    permission view_structure   = manage_structure or is_member
}

entity macro_section {
    [...]
}

entity section {
[...]
}

entity device_type {
    attribute can_pub_data  boolean
    attribute can_issue_cmd boolean
    // ... other capability attributes

    permission allowed_pub_data   = can_pub_data
    permission allowed_issue_cmd  = can_issue_cmd
    // ... other permissions derived from attributes
}

entity device {
    relation section    @section
    relation type       @device_type
    relation controller @user | @user_group#member

    permission publish_data = type.can_pub_data
    permission subscribe_cmd = type.can_sub_cmd
    permission receive_cmd  = section.operate_devices or controller
    permission read_data    = section.view_data
}
```

Code A.3: Permify schema example

```
reqBody := PermifyCheckRequest{
  Metadata: PermifyMetadata{
    SnapToken:     "",
    SchemaVersion: "",
    Depth:         100,
  },
  Entity: PermifyEntity{
    Type: entityType,
    ID:   entityID,
  },

  Permission: permission,

  Subject: PermifySubject{
    Type: subjectType,
    ID:   subjectID,
  },
}
```

Code A.4: Example of a permission check request to Permify

```yaml
vernemq2:
  image: vernemq-from-source:latest
  container_name: vernemq2
  labels:
    - "traefik.enable=true"

    # MQTT TCP (port 1883) - Load balanced
    - "traefik.tcp.routers.mqtt.entrypoints=mqtt"
    - "traefik.tcp.routers.mqtt.rule=HostSNI(`*`)"
    - "traefik.tcp.routers.mqtt.service=mqtt-backend"

    # MQTT TLS (:8883)
    - "traefik.tcp.routers.mqtt-secure.entrypoints=mqtts"
    - "traefik.tcp.routers.mqtt-secure.rule=HostSNI(`*`)"
    - "traefik.tcp.routers.mqtt-secure.tls=true"
    - "traefik.tcp.routers.mqtt-secure.tls.certresolver=letsencrypt
    - "traefik.tcp.routers.mqtt-secure.service=mqtt-backend"

  environment:
    - DOCKER_VERNEMQ_ALLOW_ANONYMOUS=off
    # Disable built-in ACLs to use only the webhook authentication/authorization
    - DOCKER_VERNEMQ_PLUGINS__VMQ_ACL=off
    # plugin webhooks managed via vernemq.confs
    - DOCKER_VERNEMQ_PLUGINS__VMQ_WEBHOOKS=on
    # Webhooks pointing to auth-service
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__JWT_AUTH__HOOK=auth_on_register
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__JWT_AUTH__ENDPOINT=http://auth-service:8080/auth_on_register
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__JWT_PUBLISH__HOOK=auth_on_publish
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__JWT_PUBLISH__ENDPOINT=http://auth-service:8080/auth_on_publish
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__JWT_SUBSCRIBE__HOOK=auth_on_subscribe
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__JWT_SUBSCRIBE__ENDPOINT=http://auth-service:8080/auth_on_subscribe
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__ON_CLIENT_OFFLINE__HOOK=on_client_offline
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__ON_CLIENT_OFFLINE__ENDPOINT=http://auth-service:8080/on_client_offline
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__ON_CLIENT_GONE__HOOK=on_client_gone
    - DOCKER_VERNEMQ_VMQ_WEBHOOKS__ON_CLIENT_GONE__ENDPOINT=http://auth-service:8080/on_client_gone

    # auto-join cluster to first node
    - DOCKER_VERNEMQ_DISCOVERY_NODE=${VERNEMQ_IP_1}
  networks:
    vernemq-net:
      ipv4_address: ${VERNEMQ_IP_2}
  depends_on:
    - vernemq1
```

Code A.5: VerneMQ configuration in Docker Compose - relevant parts

```go
func main() {
  [...]

  r.POST("/auth_on_register", authOnRegister)
  r.POST("/auth_on_publish", authOnPublish)
  r.POST("/auth_on_subscribe", authOnSubscribe)
  r.POST("/on_client_offline", onClientOffline)
  r.POST("/on_client_gone", onClientGone)

  [...]
}
```

Code A.6: Auth service - webhook endpoints handlers

```json
{
  "name": "IOT_INCOMING_DATA",
  "subjects": ["incoming.mqtt.>", "incoming.coap.>", "outcoming.coap.>", "outcoming.mqtt.>"],
  "retention": "limits",
  "max_consumers": -1,
  "max_msgs": 100000,
  "max_bytes": -1,
  "max_age": 86400000000000,
  "storage": "file",
  "num_replicas": 3
}
```

Code A.7: NATS JetStream stream configuration

```go
dtlsConfig := &piondtls.Config{
  PSK: func(hint []byte) ([]byte, error) {
    deviceHint := string(hint)
    log.Printf("DTLS Handshake: Authenticating device '%s'", deviceHint)

    key, err := pskStore.GetKey(hint)
    if err != nil {
      log.Printf("Authentication failed for device '%s': %v", deviceHint, err)
      return nil, err
    }

    log.Printf("✓ Device '%s' authenticated", deviceHint)
    return key, nil
  },
  PSKIdentityHint: []byte("coap-bridge-server"),
  CipherSuites: []piondtls.CipherSuiteID{
    piondtls.TLS_PSK_WITH_AES_128_CCM_8,
    piondtls.TLS_PSK_WITH_AES_128_GCM_SHA256,
  },
}
```

Code A.8: DTLS configuration with PSK callback function

```go
type InputPayload struct {
    DeviceID string          `json:"device"`
    Type     string          `json:"type"`
    Ts       int64           `json:"ts"`
    Data     json.RawMessage `json:"data"`
    TenantID string          `json:"tenant_id"`

    [...]

    tableName := tenantTableName(payload.TenantID)
    // Create the table if it doesn't exist (only for 1st message)
    if res := db.Table(tableName).Create(&dbRecord); res.Error != nil {
      log.Printf("DB Insert Error (table %q): %v", tableName, res.Error)
      msg.Nak()
    } else {
      log.Printf("Saved to %q: device=%s tenant=%s", tableName, payload.DeviceID, payload.TenantID)
      msg.Ack()
    }
  }
}
```

Code A.9: DB Writer input payload structure

References

[1] S. Tonelli, R. Sprugnoli, and G. Moretti, "Prendo la Parola in Questo Consesso Mondiale:," 2019, [Online]. Available: https://ceur-ws.org/Vol-2481/paper71.pdf